ParkPilot

AI-Assisted Smart Parking Management System

Repository: abhinav2006-12/ai-smart-parking-system

A Note Before You Submit

This document was generated by reading every source file in the submitted project archive directly — not from a template or from assumptions about what a "typical" parking system contains. Every technical claim below is traceable to a specific file. Two things surfaced during that review that need your attention before this goes to a panel.

SECURITY ISSUE — ACTION REQUIRED

The file .env.example in the submitted project contained what appears to be a real, live Plate Recognizer API token (not a placeholder). If this file was ever committed to a public or shared GitHub repository, that token should be treated as compromised. Rotate it immediately in your Plate Recognizer account, regardless of submission timing. This document's technical content is unaffected by this issue, but it should not wait.

DOCUMENTATION DRIFT — FOR YOUR AWARENESS

The project's own README.md is out of date relative to the actual code in two material ways: (1) it still describes the app as single-device/localStorage-only, but the code now syncs to a Supabase Postgres database as the primary store; (2) it says plate detection only works under 'vercel dev', but a custom Vite middleware plugin (vite.config.js) now makes detection work under plain 'npm run dev' too. This document describes the actual current code, not the README. If a panel member reads the README separately, be ready to note that it predates the Supabase integration.

Table of Contents

1. Comprehensive Project Overview

1.1 What the project is

ParkPilot is a web-based parking lot management system built as a single-page application (SPA) in React 19 and Vite. It addresses three operational needs for a parking facility: (1) registering vehicles in and out using automatic number-plate recognition (ANPR) instead of manual data entry, (2) calculating and collecting parking fees through a UPI (India's Unified Payments Interface) QR code at checkout, and (3) giving an administrator a dashboard to monitor occupancy, revenue, and configure slot counts and pricing.

The system is designed around two distinct, separately-secured user experiences sharing one codebase and one underlying dataset:

- **Guest / Attendant interface** — the public-facing flow used at the gate: Check-In and Check-Out, both built around the camera-based plate scanner. No login is required to use this side of the app, since it represents the gate attendant's day-to-day operational tool.
- **Admin interface** — a dashboard, vehicle listing/search, and settings panel, gated behind a login screen and reachable only by navigating directly to the /admin URL (it is not linked anywhere in the visible UI).

1.2 Technology stack (verified from package.json and source)

- **React 19 + Vite** — the application framework and build tool.
- **Supabase (@supabase/supabase-js)** — hosted Postgres database and client SDK; the primary data store for vehicles, settings, and the revenue log (see 1.4).
- **Plate Recognizer Snapshot Cloud API** — third-party cloud ANPR/OCR service that reads license plate text from images.
- **A custom Vercel Serverless Function (api/anpr.js)** — a small Node backend endpoint that proxies image uploads to Plate Recognizer, so the API token never reaches the browser.
- **recharts** — charting library used for the admin dashboard's revenue area chart and slot-distribution pie chart.
- **qrcode** — generates the UPI payment QR code shown at checkout, lazy-loaded on demand.
- **oxlint** — the project's linter (used in development; not part of the runtime application).

Notably absent: there is no tesseract.js or any client-side OCR library in the current dependency list. An earlier version of this project ran OCR entirely in-browser; that approach has since been fully replaced by the cloud-based Plate Recognizer integration (see Section 1.6 for why this matters for the privacy story).

1.3 Application routing and entry points

The app does not use a routing library (no react-router). Top-level navigation is handled by a small custom hook, useRoute.js, which reads and writes window.history directly. App.jsx inspects the current URL path on load:

- **Any path other than /admin** renders the public Home screen, from which a visitor can open the Guest gate-counter flow (Check-In / Check-Out).
- **The path /admin specifically** renders the AdminLogin screen instead. There is deliberately no button, link, or menu item anywhere in the public UI that points to /admin — an operator has to know and

type the URL. This is a UI-level access-restriction decision, not a security boundary (see Section 1.8 and the Q&A on this point).

Both the AdminPanel and GuestPanel components are lazy-loaded (React.lazy + Suspense) so their code is not downloaded by a visitor who only ever looks at the public Home screen.

1.4 Data layer: Supabase-first with a localStorage fallback

This is one of the more significant architectural facts about the current codebase, and it is easy to miss if relying on the README alone (see the note at the front of this document). The hook useStore.js (src/hooks/useStore.js) governs all application state, and its actual behavior, read directly from the code, is:

1. On mount, it calls loadStoreFromSupabase() (src/lib/supabase.js), which queries three Supabase Postgres tables: settings, vehicles, and revenue_log.
2. If Supabase returns data, that becomes the live application state, and a snapshot is kept in a ref for change-diffing.
3. If Supabase has no settings row yet (first run against a fresh database), the app falls back to whatever is in the browser's localStorage (or hardcoded defaults if neither exists), and then seeds that data back into Supabase so the database becomes the source of truth going forward.
4. If the Supabase call throws (e.g. missing credentials, network failure), the app catches the error, logs it, and falls back to localStorage so the app remains usable offline or mis-configured.
5. On every subsequent state change, the app writes to localStorage immediately (a synchronous local cache) and asynchronously diffs the new state against the last-synced snapshot to push only the changed rows to Supabase (settings via upsert; vehicles and revenue_log via upsert/insert/delete as appropriate).

In short: Supabase is the system of record when configured; localStorage is a same-device cache and offline/misconfiguration fallback, not the primary store. A reviewer asking “where does the data actually live” should be told Supabase — not localStorage, despite what the README currently says.

While this load is in progress, App.jsx renders a dedicated loading screen component, VehicleLoader.jsx, which cycles through a sequence of status messages (e.g. “Connecting to secure database…”, “Syncing real-time parking spaces…”) alongside an animated SVG car illustration. This is a deliberate UX decision to mask network latency on initial load, and it is purely cosmetic — the messages do not reflect literal backend events; they are a fixed, rotating list shown for as long as the real Supabase fetch takes.

1.5 Data model

The application's central object is the “store”, with three top-level collections, mirrored between the camelCase shape used in React state and the snake_case shape used in the Supabase tables:

settings (single row)

- totalSlots, slotsByType ({standard, ev, disabled}) — capacity configuration.
- rates ({standard, ev, disabled}, each {hourly, minHours}) — the actual hourly billing rate and minimum billable hours per vehicle category.
- upiVpa, upiPayeeName — the UPI virtual payment address and payee display name used to build the checkout QR code.
- currency — stored as "INR" throughout.

vehicles (one row per parking session)

- id, number (the normalized plate text), type (standard/ev/disabled), entryTime, exitTime, status ("parked" or "completed"), fee, durationMins, entryPhoto, exitPhoto (both stored as base64 data URLs, not separate image files).

revenueLog (one row per completed, paid checkout)

- id, vehicleId, amount, date — used to drive the admin dashboard's revenue charts.

1.6 Plate recognition pipeline (ANPR)

This is the technical core of the project and is implemented as a multi-layer pipeline. Reading top to bottom:

1.6.1 Capture layer — LiveCameraCapture.jsx

Acquires the device camera directly via the browser's native getUserMedia API (no third-party camera library such as react-webcam is used). On a fixed cadence (every 1000ms, implemented as a self-rescheduling setTimeout chain rather than setInterval), it:

6. Draws the current video frame to a hidden canvas, downscaled to a maximum width of 1024px to keep the upload small and fast (the entire frame is sent, not a cropped sub-region — an earlier version of this component used a center-cropped “scanning zone” guide box, but that cropping and its on-screen aiming overlay have since been removed from the code).
7. Converts the canvas to a JPEG Blob.
8. Sends that Blob to the recognizePlate() client helper (src/lib/anpr.js), which POSTs it to this app's own /api/anpr endpoint.
9. Applies a race-condition guard: an AbortController is created per tick and aborted if a new tick starts (or the component unmounts) before the previous request resolves, paired with a monotonically increasing request-id ref so a late-arriving response can also be detected and discarded even if abort alone weren’t sufficient.
10. Runs the returned plate text through a single-reading “vote” check before locking it in and calling the parent's onDetected callback. The configured threshold (VOTES_REQUIRED) is currently 1, meaning the very first valid reading is accepted immediately, on the stated rationale (in an inline code comment) that the cloud AI's confidence is already high enough not to need cross-frame confirmation. This is a real change from an earlier 3-frame-debounce design and should be described accurately as a single-frame lock, not multi-frame voting, if asked.

1.6.2 Backend proxy layer — api/anpr.js

A Vercel Serverless Function. Its entire purpose is to keep the Plate Recognizer API token off the client. It:

- Accepts a raw image POST body (Vercel's default JSON body parser is disabled for this route).
- Re-wraps the bytes as multipart/form-data and forwards them to https://api.platerecognizer.com/v1/plate-reader/ with an Authorization: Token <value> header, the token read from process.env.PLATERECOGNIZER_TOKEN.
- Passes regions="in" to bias the recognition engine toward Indian plate formats.
- Handles three specific failure modes from the provider explicitly: HTTP 429 (rate-limited) is passed through as 429; HTTP 403 (invalid token or insufficient account credits) is translated to a generic 502

so the cause isn't leaked to the client; any other non-OK response is logged server-side and returned as a generic 502.

- Picks the highest-confidence result if Plate Recognizer's response contains more than one detected plate, and returns a small normalized {plate, confidence} JSON object to the frontend.

1.6.3 Local development shim — vite.config.js

Because /api/anpr.js is written as a Vercel Serverless Function, it would not normally run under the plain Vite dev server. The project works around this with a custom Vite plugin (vercelApiPlugin, defined directly in vite.config.js) that intercepts requests to /api/anpr during npm run dev, loads the PLATERECOGNIZER_TOKEN from the local .env file, and invokes the exact same handler function used in production — polyfilling just enough of the Vercel request/response object shape (res.status(), res.json()) for it to work. This means plate detection works under both plain npm run dev and vercel dev, contradicting the README's current claim that only vercel dev supports it.

1.6.4 Normalization and validation — src/lib/plate.js

All plate text, whether it comes from the camera, a manual photo upload, or direct keyboard typing, passes through normalizeIndianPlate(), which strips everything except A–Z and 0–9 characters (collapsing spaces, hyphens, and the newline that OCR engines sometimes insert between the two rows of a square/stacked Indian plate format) and forces uppercase. Two separate regex validators exist side by side for different purposes:

- **isLikelyValidIndianPlate** — a lenient pattern (/^[A-Z]{2}[0-9]{1,2}[A-Z]{1,3}[0-9]{1,4}$/) that tolerates older plate formats with a single-digit district/RTO code, used only to show a soft, non-blocking “doesn't quite match the standard format” warning under the text field.
- **isStrictIndianPlate** — the canonical, fully-formed 10-character pattern (/^[A-Z]{2}[0-9]{2}[A-Z]{1,3}[0-9]{4}$/), used to gate automatic actions: triggering an automatic Check-Out lookup, and flagging an automatic Check-In duplicate warning. Loosely-formed or partial text never auto-triggers these actions — only a complete, strictly-formatted plate does.

1.6.5 Manual fallback — PlateCapture.jsx

A segmented toggle lets the operator switch between “Live AI Camera” and “Manual Upload” modes (for poor lighting or hardware failure). Manual mode accepts a photo file via a standard <input type="file"> (with the capture="environment" attribute, which on mobile browsers opens the device camera app directly rather than a file picker) and sends that file through the identical recognizePlate() / /api/anpr pipeline used by the live camera — there is no separate, duplicate OCR code path for manual uploads; both ultimately hit the same backend proxy and the same Plate Recognizer account.

1.7 Check-In workflow (CheckInFlow.jsx)

The attendant first selects a vehicle category (Standard / EV / Disabled) from three buttons, each showing live remaining-slot counts and disabling itself if that category is full. The plate capture component sits below this selector. There are, in the current code, two distinct ways a check-in can be completed:

- **Automatic path** — the PlateCapture component's onDetected callback fires with both plate text and a non-null photo data URL whenever a detection completes (this is true for both the live camera AND a successful manual photo upload, since both pass a photo through). The moment that happens, CheckInFlow immediately validates slot availability and duplicate-parked status, and if both checks pass, creates the vehicle record and writes it to the store without the operator clicking anything. A full-screen frosted-glass success overlay (“Auto Checked In”) then appears showing the plate, type,

and entry time, with a 10-second countdown after which it dismisses itself and resets the scanner (bumping a captureSessionId key to force PlateCapture/LiveCameraCapture to fully remount, clearing internal vote-buffer state) ready for the next vehicle.

- **Manual path** — the plate number is also always shown in an editable text field underneath the capture component. The attendant can type or correct it directly, and click “Confirm Check-In”, which runs the same slot/duplicate validation before creating the record. This button is disabled while a live duplicate-vehicle warning is showing.

Additionally, the auto-check-in overlay includes an “Edit Manually” escape hatch: clicking it deletes the just-created vehicle record from the store and drops the plate/photo back into the editable manual form, in case the automatically-detected text was wrong.

A right-hand panel on the Check-In screen renders PriceChartCard inline — see Section 1.9 for an important clarification about what this chart actually represents.

1.8 Check-Out workflow (CheckOutFlow.jsx)

Structurally similar to Check-In, but its automatic behavior is a lookup rather than a record creation. A useEffect watches the plate text field; the instant its value becomes a strict-format, fully-formed plate (whether typed or scanned), it automatically calls tryMatch(), which searches store.vehicles for a vehicle with status: "parked" and a matching number — no manual “Find” click is required, though that button still exists for manual retry. If found, the app computes:

- Duration parked, in minutes, from entryTime to the current time.
- Billable hours: the elapsed time rounded UP to the next whole hour, with a floor of the vehicle type's configured minHours (so a 10-minute stay for a Standard vehicle with minHours: 1 still bills 1 full hour).
- Fee: billable hours × that vehicle type's configured hourly rate (from store.settings.rates) — not a flat, hardcoded amount.

A UPI deep-link string is then built (src/lib/format.js, buildUpiUri) in the form upi://pay?pa=<vpa>&pn=<payee>&am=<fee>&cu=INR&tn=Parking%20fee%20<plate>, using the operator-configured VPA and payee name from Settings, and rendered as a scannable QR code image via the qrcode library. A “Simulate Payment Success” button (there is no real payment-gateway integration or webhook — this is explicitly a simulated/manual confirmation step) marks the vehicle status as completed, records the fee, exit time, and exit photo, and appends an entry to the revenue log. A “Scan Next Vehicle” button on the success screen resets local state and bumps the same captureSessionId remount key described above.

1.9 Important clarification: the “Dynamic Pricing” chart is illustrative, not live

PriceChartOverlay.jsx exports a component, PriceChartCard, that renders a polished, glassmorphic, tabbed area chart labelled “Parking Rates” with the subtitle “Real-time dynamic hourly pricing based on time of day and vehicle demand.” It is shown inline on the Check-In screen. Read literally from the source code, this component's chart data is a hardcoded constant array, DEFAULT_PRICING_DATA, defined inside the component file itself, with eight fixed time-of-day data points per vehicle type. It is not computed from store.settings.rates (the real, configurable rates that actually drive billing elsewhere in the app), it does not read live demand or occupancy, and changing a rate in Settings has no effect on this chart. If asked directly: this chart is a visual mockup of what a future dynamic-pricing feature could look like, styled to production quality, but it is not functionally wired to the app's real billing logic. This distinction matters and should be stated plainly rather than allowed to imply a live feature that does not exist.

1.10 Admin Panel

Reachable only via the hidden /admin URL and gated by AdminLogin.jsx. The current login screen, despite its visual design implying enterprise-grade authentication (an “ADMIN GATEWAY” badge, a simulated 1.2-second “Authenticating…” delay, a 5-attempt lockout with a 30-second countdown), performs a hardcoded client-side string comparison: it checks the entered email against the literal string "parkpilot@gmail.com" and the password against the literal string "parkpilot2025", both visible directly in the AdminLogin.jsx source. The 5-attempt lockout state lives only in React component state and resets completely on page reload, so it provides no protection against a determined or even mildly persistent attempt. This is explicitly flagged as a placeholder, not production authentication, both in the project's own README and in this document's Q&A defense section.

Once authenticated, AdminPanel.jsx presents a collapsible sidebar (collapsed by default on narrow viewports, toggled by a hamburger button) with three tabs:

- **Dashboard (DashboardTab.jsx)** — six summary stat cards (Total Vehicles, Active Parking, Left Today, Empty Slots, Revenue Today, Revenue Yesterday), a 7-day revenue area chart, and an interactive donut chart of slot allocation by vehicle type (clicking a segment updates the center label to show that segment's count).
- **Vehicle Listing (VehicleListingTab.jsx)** — a searchable (by plate number), filterable (by status and vehicle type), paginated (8 rows per page) table of every vehicle record, showing entry/exit time, duration, fee, and status.
- **Settings (SettingsTab.jsx)** — editable forms for per-type slot counts (with the total computed automatically as their sum), per-type hourly rate and minimum billable hours, and the UPI VPA / payee name used for checkout QR codes. A “Save Settings” button writes the whole settings object back to the store (and from there, to Supabase, per Section 1.4).

1.11 Theming and visual design

A light/dark theme toggle (useTheme.js) persists the choice to localStorage and defaults to the operating system's prefers-color-scheme on first visit. Theme is applied via a data-theme attribute on the document root, with the entire palette (background, surface, border, accent, success/warning/danger colors, etc.) defined as CSS custom properties in index.css, so components reference variables like var(--accent) rather than hardcoded hex values. AmbientBackground.jsx renders a fixed, decorative, non-interactive layer of softly blurred radial color blobs plus a faint grid pattern, purely as a visual backdrop — it has no functional role.

1.12 Notable engineering practices worth highlighting to a technical panel

- Camera resource cleanup: LiveCameraCapture explicitly calls .stop() on every individual MediaStreamTrack on unmount (not just clearing the <video> element's srcObject), which is the step that actually releases the camera hardware and turns off the device's camera-active indicator light — a common real-world bug in webcam-based React components is forgetting this and leaving the camera running after the component disappears.
- Race-condition handling in the scan loop: both an AbortController and a separate monotonically-increasing request-id ref guard against a slow, stale network response overwriting a result from a newer, faster one — a realistic risk at a 1-request-per-second cadence against a third-party API with variable latency.

- Forced remount via key prop: resetting local component state (like plateNumber) back to empty does NOT, by itself, reset a child component's internal state in React; the codebase deliberately uses a bumped captureSessionId as a key on <PlateCapture> specifically to force a true unmount/remount between vehicles, which is what actually clears the camera component's internal vote buffer between sessions.
- Coordinate-space awareness in frame capture: the capture canvas logic accounts for the difference between a video element's native pixel resolution and its on-screen CSS-rendered size, which differ on most real device/browser combinations — a frequent source of misaligned crops in naive camera-capture implementations.

2. Anticipated Evaluation Questions & Defense

The questions below are organized into five categories a corporate evaluation panel would realistically draw from: Architecture & Technical Design, AI/ANPR Implementation, Data & Backend, Security & Production-Readiness, and Business/Product Rationale. Every answer is grounded in the actual source code reviewed in Section 1 — where a feature has a real limitation, the answer says so directly rather than overstating the implementation. This is intentional: a reviewer who probes a confidently-overstated claim and finds it false does more damage to the project's credibility than an honest, well-reasoned acknowledgment of a known limitation.

2.1 Architecture & Technical Design

Q1. Walk me through the overall architecture of this system.

A: It's a React 19 single-page application built with Vite, with two cooperating backend services rather than one monolithic backend. Application data (vehicle records, settings, revenue) lives in a Supabase-hosted Postgres database, accessed through the Supabase JS client, with a localStorage cache as an offline/fallback layer. The second backend piece is a small Vercel Serverless Function (api/anpr.js) whose only job is to securely proxy plate images to the third-party Plate Recognizer Snapshot Cloud API, keeping that API's secret token off the client. The frontend itself has no client-side router library; navigation between the public Home screen, the Guest gate-counter flow, and the hidden Admin panel is handled by a small custom hook reading the browser's History API directly.

Q2. Why did you choose Supabase instead of building a custom backend with something like Express or Django?

A: Supabase gives a hosted Postgres database, a generated REST-like client API, and authentication primitives without writing and operating a separate server process. For a project of this scope — a handful of tables, no complex business logic beyond what already lives in the frontend's fee calculation — a custom backend would mean writing and maintaining equivalent CRUD endpoints for no functional gain. The trade-off, worth being upfront about, is that some logic that would normally live behind a server boundary (such as the Supabase Row Level Security policy configuration) is shaped by client-side calls rather than a dedicated API layer; that is a reasonable choice for this project's current scale, not a universal best practice.

Q3. Why is there no React Router, Redux, or similar standard library in this project?

A: The project deliberately favors small, purpose-built code over framework dependencies where the surface area is small enough to justify it. There are exactly two real “routes” in the whole app (“everything” and “/admin”), so a 30-line custom hook (useRoute.js) reading window.history covers the requirement without pulling in a routing library's full feature set. Similarly, application state is a single object with one update function (useStore.js, using plain React state plus useEffect for persistence), which is simple enough that a state-management library like Redux would add boilerplate without solving a problem this codebase actually has. This is a judgment call appropriate to the project's current size; it would be revisited if the route count or state-update complexity grew substantially.

Q4. How does client-side routing handle a direct browser refresh on /admin in production?

A: Because there's no server-rendered routing, a static host needs to be told to serve index.html for any path so the client-side router can take over and read the URL itself — otherwise a hard refresh on /admin would hit the host looking for a literal file at that path and 404. The project includes vercel.json (a catch-all rewrite to index.html) for Vercel deployments and a public/_redirects file for Netlify, covering both supported hosting paths.

Q5. Explain the lazy-loading strategy used in App.jsx and why it matters.

A: AdminPanel and GuestPanel are both loaded via React.lazy() wrapped in Suspense, rather than imported eagerly at the top of the file. This means their JavaScript (and, transitively, their dependencies — recharts for the admin dashboard, qrcode for checkout, the entire camera/ANPR pipeline for guest flows) is only downloaded by the browser once a visitor actually navigates into one of those areas. A first-time visitor who only loads the public Home screen never pays the bandwidth or parse cost for the Admin dashboard's charting library, for example. This is a standard code-splitting technique and shows up directly in the production build output as separate chunk files.

Q6. What is the AmbientBackground component, and why does it exist?

A: It's a purely decorative, non-interactive layer of softly blurred radial gradients and a faint grid line pattern, rendered behind all page content via fixed CSS positioning. It has no state, no props that affect behavior, and no functional role — it exists solely to make the flat, mostly-white design language feel less sterile. It's worth being direct that this is cosmetic, not a feature, if a panel asks what it does.

Q7. How is theming (light/dark mode) implemented, and where is the choice persisted?

A: A custom hook, useTheme.js, holds the current theme in React state, writes it to a data-theme attribute on the document's root <html> element, and persists the choice to localStorage so it survives a reload. On first visit (no stored preference), it defaults to whatever the OS reports via the prefers-color-scheme media query. Every component then styles itself using CSS custom properties (e.g. var(--accent), var(--surface)) defined twice in index.css — once for the default palette and once scoped under [data-theme="dark"] — so no component needs its own conditional theme logic; the browser resolves the correct variable value automatically based on that root attribute.

Q8. The project structure shows both a /api directory and normal Vite client code. How do these two runtimes coexist locally during development?

A: This required a specific workaround, implemented in vite.config.js. Vercel Serverless Functions follow a Node.js request/response convention that the plain Vite dev server doesn't natively serve. The project defines a custom Vite plugin that intercepts any request to /api/anpr during npm run dev, loads the required environment variable from the local .env file, constructs minimal polyfilled res.status()/res.json() methods matching what the real handler expects, and then calls the exact same exported handler function that Vercel would call in production. This means there is only one implementation of the ANPR proxy logic, exercised identically in both local development and production — not a separate mock used only for local testing.

2.2 AI / ANPR Implementation

Q9. How does the automatic plate detection actually work, end to end?

A: The live camera component captures a frame from the device camera roughly once per second using the browser's native getUserMedia API. Each captured frame is drawn to a canvas, downscaled to a maximum

width of 1024 pixels, and converted to a JPEG image blob. That blob is sent to this app's own backend endpoint, /api/anpr, which forwards it (with the secret API token attached server-side) to Plate Recognizer's cloud ANPR service. Plate Recognizer returns the detected plate text and a confidence score; the app normalizes that text (uppercase, strip separators) and displays it. If it's a complete, well-formed plate, downstream logic (Check-In or Check-Out) can act on it automatically.

Q10. Is the plate detection done on-device or in the cloud? This affects our data residency requirements.

A: It is cloud-based, not on-device, and this is a meaningful architectural fact worth stating plainly rather than glossing over: every captured frame (camera) and every manually uploaded photo is sent over the network to Plate Recognizer's servers for processing. An earlier iteration of this project ran OCR entirely client-side via tesseract.js with a genuine “no photo ever leaves the browser” guarantee; that approach has been fully replaced. If data residency is a hard requirement for your deployment, that needs to be evaluated against Plate Recognizer's own data handling and retention policy, since this application has no control over what happens to an image once it leaves the /api/anpr proxy.

Q11. Why use a third-party paid API instead of an open-source OCR engine?

A: Open-source, in-browser OCR (the project's own earlier approach, using tesseract.js) is general-purpose text recognition, not purpose-built for license plates; its accuracy on plate-specific challenges — oblique camera angles, partial occlusion, varying plate fonts and reflective surfaces, two-row/stacked plate layouts common on Indian SUVs — is materially lower than a model trained specifically for ANPR. Plate Recognizer is a commercial service trained specifically on that problem and supports region-specific tuning (this project passes regions="in" to bias it toward Indian plate formats). The trade-off accepted in exchange is the items above: network dependency, a per-call cost, a rate limit, and the photo leaving the browser.

Q12. What happens if there's no internet connection, or the Plate Recognizer service is down?

A: Plate detection fails outright in that scenario — it is not graceful degradation to a lower-accuracy local fallback, because there currently is no local OCR fallback in the codebase. The backend proxy (api/anpr.js) returns a clear error response (502 for upstream failures, 500 for a missing API token configuration) rather than crashing silently, and the frontend surfaces that as a visible “ANPR API error” message under the camera view. The operator can still complete a Check-In or Check-Out by typing the plate number manually into the always-present text field; that manual path does not depend on the camera or the ANPR service at all.

Q13. Explain the debouncing/voting logic for plate detection. Does it require multiple consistent readings before locking in a result?

A: It's important to be precise here rather than describe an earlier design: the current code's voting threshold (VOTES_REQUIRED) is set to 1, meaning a single valid reading from the AI is accepted and locked in immediately, with an inline comment in the source stating the rationale that the cloud AI's confidence is already high enough that cross-frame confirmation is treated as unnecessary overhead. The voting mechanism itself (a small rolling buffer compared against itself) is still present in the code and could be raised back to require multiple consecutive identical readings by changing that one constant, but as shipped, it does not wait for repeated confirmation across frames.

Q14. How do you prevent two browser tabs, or two rapid back-to-back frames, from both submitting a plate detection at the same time and causing a race condition?

A: Within a single capture session, two mechanisms work together: an AbortController is created fresh for every capture tick and explicitly aborted the moment a new tick begins (or the component unmounts), which cancels the in-flight network request itself; separately, a monotonically incrementing request-id is captured in a closure for each tick, and checked again after each await, so that even in an edge case where a request technically completes despite being superseded, its result is discarded rather than applied to the UI. This combination protects against a slow earlier response landing after a faster, more recent one. Cross-tab or cross-device race conditions (two attendants somehow processing the same physical car at once) are a separate, broader concern not specifically handled by this mechanism — the duplicate-parked-vehicle check in Check-In (comparing against store.vehicles) is the relevant safeguard there, operating at the data layer rather than the capture layer.

Q15. Does the system crop the image to just the license plate before sending it for recognition, to reduce bandwidth and improve accuracy?

A: Not in the current code. An earlier version of this component implemented a center-cropped “scanning zone” with an on-screen aiming-guide overlay so only that cropped region was sent for recognition. That cropping logic and its overlay have since been removed; the current implementation downscales and sends the entire camera frame (resized to a maximum width of 1024px) and relies on Plate Recognizer's own vehicle/plate detection to locate the plate within the full image. This is a legitimate design choice — Plate Recognizer's API is built to find a plate anywhere in a vehicle photo, not just in a pre-cropped region — but it does mean more image data is transmitted per request than the cropped approach would send.

Q16. What's the difference between the two regex patterns in plate.js, and why have two?

A: They serve genuinely different purposes and using the wrong one in the wrong place would cause real bugs. isLikelyValidIndianPlate is intentionally lenient — it accepts both one- and two-digit RTO/district codes, which reflects that genuinely valid older-format plates exist with a single digit there. It's used only to show a soft, non-blocking visual warning under the input field (“doesn't quite match the standard format”) so an attendant double-checks an OCR misread without being blocked from proceeding. isStrictIndianPlate requires the full canonical 10-character format and is used specifically to gate automatic actions — triggering an automatic Check-Out lookup, or flagging a Check-In duplicate warning — because those actions should not fire on a half-typed or partially-OCR'd plate string.

Q17. How does the system handle the two-row/stacked plate format common on Indian SUVs and commercial vehicles?

A: The normalization function, normalizeIndianPlate, strips every character that is not A–Z or 0–9, which collapses the newline character that OCR engines often insert between the top and bottom rows of a stacked plate, along with spaces and hyphens, into one continuous uppercase string. This handles the text-formatting side of a stacked plate cleanly. Whether the underlying Plate Recognizer model itself reads a stacked plate's two rows correctly in the first place is governed by their model, not this application's code — this project's normalization step operates on whatever text string comes back from that model.

Q18. Why does manual photo upload also trigger automatic check-in, not just the live camera?

A: This is a real, verifiable behavior worth being precise about: the automatic check-in logic in CheckInFlow.jsx is gated on whether the onDetected callback received a non-null photo, not on which capture mode produced it. A successful manual upload also passes a non-null photo data URL through that same callback, so it also auto-submits the check-in, exactly like a live-camera detection would. This is consistent behavior by design — the manual upload path is meant as a full equivalent fallback for the

camera, not a degraded one — but it does mean a panel member testing manual upload should expect the same auto-submit behavior as the camera, not a separate manual-confirm-only flow.

2.3 Data & Backend

Q19. Where exactly is data stored, and what happens on a fresh deployment with an empty database?

A: The primary store is Supabase Postgres, across three tables (settings, vehicles, revenue_log), accessed through useStore.js and src/lib/supabase.js. On first load against a brand-new, empty database (no settings row yet), the app does not fail — it falls back to whatever exists in the browser's localStorage, or to a hardcoded default configuration if even that is empty (50 total slots split 30/12/8 across standard/EV/disabled, with default hourly rates), and then writes that starting configuration back into Supabase so the database becomes authoritative from that point forward.

Q20. What happens if Supabase is unreachable at runtime — does the app crash?

A: No. The load function wraps its Supabase calls in a try/catch; on any thrown error (network failure, misconfigured credentials, etc.) it logs the error to the console and falls back to loading from localStorage instead, so the app remains usable on that device using whatever was last cached locally. Writes follow the same pattern — a failed sync to Supabase is caught and logged, not allowed to crash the UI, though it does mean that device's changes won't propagate to other devices until connectivity is restored.

Q21. Is the Supabase sync real-time across multiple devices, or does each device only see its own local changes?

A: Based on the actual code, sync is not real-time/subscription-based — there is no Supabase real-time channel subscription in the codebase. Data is fetched once on mount and written back on every local state change; a second device open at the same time would not automatically see updates from the first device without a manual refresh (which re-runs the initial load effect on next mount). This is a meaningful gap to be upfront about if multi-terminal, simultaneous-use deployment (e.g. two gate attendants on separate tablets at the same time) is a requirement — the current implementation is closer to “each session syncs its changes up and pulls fresh on (re)load” than true live multi-user collaboration.

Q22. How does the app decide whether to UPSERT, INSERT, or DELETE when syncing to Supabase?

A: syncStoreToSupabase performs a diff between the new in-memory state and a snapshot of the last-synced state. For settings (a single row), any change at all triggers a full upsert. For vehicles and revenueLog, it builds a Map of the old records keyed by id, then for each new record either includes it in an upsert batch (if it's new or has changed) or skips it (if unchanged); it separately computes which old ids are no longer present in the new state and issues a delete for exactly those. This minimizes the number of database writes rather than rewriting every row on every change.

Q23. Why store vehicle entry/exit photos as base64 strings in the database rather than uploading them to a file storage bucket?

A: That is how the current code is implemented — entryPhoto and exitPhoto are stored as base64-encoded data URLs directly in the vehicles table's entry_photo/exit_photo columns, not as references to files in Supabase Storage or any other object store. This is the simplest approach to implement and keeps photo retrieval a single query with no separate storage API calls, but it is worth being direct about the trade-off if

asked: base64-encoded images inflate row size substantially compared to their binary form, which has real implications for database storage costs and query/transfer performance as the vehicle history grows. Migrating to Supabase Storage (storing a reference URL instead of the raw image data) would be a reasonable optimization at scale, but is not what's implemented today.

Q24. How is the parking fee actually calculated? Walk through the formula.

A: On Check-Out, durationMinutes(entryTime, now) computes elapsed minutes since check-in. That figure is converted to billable hours by rounding up to the next whole hour (Math.ceil(minutes / 60)) and then taking the maximum of that value and the vehicle type's configured minimum billable hours (minHours) — so, for example, a 10-minute Standard-vehicle stay with minHours: 1 still bills exactly 1 hour, not a fraction of one. The fee is then billable hours multiplied by that vehicle type's configured hourly rate, both values read live from store.settings.rates, not hardcoded. This means changing a rate in the Settings tab immediately changes how new checkouts are billed.

Q25. Is the UPI QR code payment a real, working payment integration?

A: It generates a real, standards-compliant UPI deep link (upi://pay?pa=...&pn=...&am=...&cu=INR&tn=...) using the operator's configured VPA and payee name, and renders it as a genuine, scannable QR code image via the qrcode library — scanning it with an actual UPI app (PhonePe, Google Pay, etc.) would attempt a real payment to whatever VPA is configured in Settings. What it does not include is any payment confirmation, webhook, or callback mechanism to verify that the payment was actually completed. The “Simulate Payment Success” button is exactly what its label says: a manual attendant action confirming receipt, with no automated verification that money actually moved. This should be described accurately as QR generation with manual confirmation, not an end-to-end payment gateway integration.

2.4 Security & Production-Readiness

Q26. Is the admin login a real authentication system?

A: No, and this should be stated plainly rather than implied otherwise: AdminLogin.jsx checks the entered email and password against two hardcoded literal strings directly in the component's source code ("parkpilot@gmail.com" / "parkpilot2025"). There is no backend authentication call, no password hashing, no session token, and no integration with Supabase Auth despite Supabase already being used elsewhere in the project for data. The screen's polished visual design — a simulated authentication delay, a 5-attempt lockout with a 30-second countdown — can give a false impression of real security; the lockout itself lives only in transient React component state and resets completely on a page reload, so it does not meaningfully prevent repeated attempts. This is explicitly documented as a placeholder in the project's own README and should be replaced with real authentication (e.g. Supabase Auth, which is already a project dependency) before any production use with real customer or payment data.

Q27. If the admin URL is hidden and not linked anywhere, isn't that a security measure?

A: It raises the bar for casual discovery, but it is access obscurity, not access control, and the project's own README states this directly. Anyone who guesses, is told, or finds the /admin path in the page source, browser history, or this very document still reaches a login screen protected only by the hardcoded credentials described above. The two should not be conflated when discussing the project's security posture: hiding the URL stops accidental discovery; it does nothing against anyone who specifically wants in.

Q28. How is the third-party API token protected from exposure to end users?

A: The Plate Recognizer API token is read exclusively from a server-side environment variable, PLATERECOGNIZER_TOKEN, inside api/anpr.js — a function that runs on Vercel's servers, never in the browser. The token is never included in any client-side JavaScript bundle, never sent to the browser in any response, and is not present in any file that ships to the client. The frontend only ever talks to this project's own /api/anpr endpoint, never directly to Plate Recognizer, so it has no way to learn the token even by inspecting network traffic in the browser's developer tools.

Q29. Were there any actual security issues found during review of this codebase?

A: Yes, one concrete issue, and it should be raised directly rather than omitted: the project's .env.example file — a file whose entire purpose is to be safe to commit to version control as a template — was found to contain what appears to be a real, live Plate Recognizer API token rather than a placeholder value. If that file was ever pushed to a shared or public GitHub repository, that token must be treated as compromised and rotated immediately in the Plate Recognizer account, independent of this submission's timing. This has been corrected in the reviewed copy of the project (replaced with a generic placeholder), but the live token may still exist in the actual repository's git history unless it is separately scrubbed or the key is rotated.

Q30. What is the current rate limit on plate recognition, and what happens if it's exceeded?

A: Plate Recognizer's documented Free Trial tier allows 1 lookup per second; a paid Cloud subscription raises that to 8 per second. The live camera's auto-capture loop runs at exactly 1 frame per second, which is right at the free tier's ceiling with no margin — occasional 429 (rate-limited) responses should be expected on the free plan, especially if any latency causes two requests to overlap. The backend proxy passes a 429 through explicitly rather than masking it, and the frontend surfaces it as a visible error message rather than failing silently. A paid plan removes this constraint in practice.

Q31. Does the application have any automated tests?

A: Based on the submitted files, no — there is no test runner configuration, no test files, and no test script beyond the linter (oxlint) in package.json's scripts. This is a genuine gap for a system that handles billing calculations and vehicle record state, and it's appropriate to acknowledge it directly: the fee calculation logic, plate normalization/validation regexes, and the Supabase diff-sync logic in particular would all benefit from unit test coverage given they're pure, testable functions with clear expected inputs and outputs.

Q32. Is there any rate limiting, CORS configuration, or abuse protection on the /api/anpr endpoint itself?

A: The endpoint enforces only that the HTTP method is POST; there is no application-level rate limiting, authentication, or request-origin restriction on /api/anpr beyond what Plate Recognizer's own API enforces on the upstream side. In its current form, anyone who can reach the deployed app's domain can call this endpoint directly and consume the configured Plate Recognizer account's quota, since the endpoint itself doesn't check who is calling it. Adding origin checks or a lightweight rate limit at this proxy layer would be a reasonable hardening step before a public, unauthenticated production deployment.

2.5 Business & Product Rationale

Q33. What real-world problem does this system solve, and for whom?

A: It targets the operational pain points of a manually-run parking facility: attendants hand-writing or hand-typing plate numbers (slow, and a frequent source of billing/record errors), no live visibility into occupancy or revenue for a facility owner/manager, and cash-only or manual payment collection at exit. ParkPilot replaces manual plate entry with camera-based capture, computes fees automatically and consistently from a single configured rate table rather than attendant memory or a paper rate card, and gives an owner or manager a dashboard view of occupancy and revenue trends without needing to be physically present.

Q34. What is the actual cost structure of running this system in production?

A: Three recurring costs exist beyond standard web hosting: a Supabase project (free tier covers a meaningful amount of usage for a single facility; paid tiers scale with database size and bandwidth), a Plate Recognizer Snapshot Cloud subscription (free trial is 1 lookup/second, which is exactly the camera's scan cadence with no margin; a paid plan at 8 lookups/second would be the realistic production tier for a live gate), and Vercel hosting for the serverless function and static site (free tier is generally sufficient for this app's traffic profile unless usage is very high). None of these costs are currently surfaced or estimated anywhere in the project's own documentation, which would be a reasonable addition before a cost-sensitive stakeholder evaluates it.

Q35. How does this system handle multiple simultaneous gate attendants or multiple physical entry points?

A: The codebase doesn't currently model “multiple gates” or “multiple attendants” as distinct concepts — every device running the app's Guest flow reads and writes the same shared Supabase-backed vehicle/settings data, so multiple devices can be used concurrently in the sense that they all see (after a sync) the same underlying records. However, as noted in Section 2.3, there is no real-time subscription mechanism, so two attendants working at the same moment on separate devices would not see each other's just-completed check-ins until each device's own next sync/reload. For a single-gate, single-attendant-at-a-time deployment this is a non-issue; for a larger facility with several simultaneously-staffed entry points, this would need to be addressed before relying on it as the single source of truth in real time.

Q36. What would need to change to take this from a prototype/demo to a production deployment a real paying customer could rely on?

A: Speaking plainly, in priority order based on actual risk: (1) replace the hardcoded admin credentials with real authentication — Supabase Auth is already a project dependency and would be a natural fit; (2) move to a paid Plate Recognizer tier to remove the 1-request/second ceiling that the free tier shares exactly with the live camera's scan rate; (3) add real payment verification (a payment gateway webhook) rather than the current manual “Simulate Payment Success” confirmation, since that step currently has no way to detect a no-show or failed payment; (4) add Supabase real-time subscriptions if multi-device simultaneous use is a requirement; (5) add automated test coverage, particularly around the fee calculation and plate validation logic; (6) rotate the Plate Recognizer API token that was found exposed in .env.example, and review the wider git history for similar exposure.

Q37. Why support three separate vehicle categories (Standard, EV, Disabled) rather than a single uniform rate?

A: This reflects a genuine, common real-world parking-facility requirement: many jurisdictions and many private facility policies mandate reserved capacity and preferential (often lower) rates for disabled-accessible parking, and a growing number of facilities offer discounted or dedicated EV charging-adjacent parking to encourage electric vehicle adoption. Modeling these as distinct categories — each with its own

slot allocation and its own rate/minimum-hours configuration — lets a facility operator set policy for each independently rather than forcing one rate to fit all vehicle types.

Q38. The pricing chart on the Check-In screen implies dynamic, demand-based pricing. Is that a real feature, or a roadmap item?

A: To be direct about this, since it's an easy point to misrepresent: as implemented today, it is a polished visual mockup, not a live feature. Its chart data is a hardcoded constant defined in the component file, entirely disconnected from the real, configurable rates in Settings that actually drive billing. It should be described to a panel as a demonstration of what a future time-of-day or demand-based dynamic pricing feature could look like visually, not as a currently-functioning capability. Being upfront about this distinction is preferable to a panel member discovering it themselves by checking whether changing a Settings rate affects the chart (it doesn't).

3. Summary: Honest Assessment for the Defense

A strong defense of this project does not require claiming it is more finished than it is. The strongest position available, based on direct review of the actual code, is this:

3.1 What is genuinely well-executed

- A complete, working end-to-end flow from camera capture through cloud-based plate recognition through automatic fee calculation through UPI QR generation — not a mockup of these steps, but real, functioning logic for each one.
- Careful, specific attention to real engineering failure modes that are easy to get wrong in camera-based React components: explicit MediaStreamTrack cleanup, dual race-condition guards on the recognition request loop, and deliberate component remounting (via a bumped key) to reset internal state between vehicles.
- A sensible default-then-fallback data architecture (Supabase primary, localStorage fallback) that keeps the app usable even when the cloud database is briefly unreachable or not yet configured.
- A real, secure secret-handling pattern for the third-party API token — it is never exposed to the client, which is the correct architecture even though one example-config file slipped and needs fixing.
- Thoughtful UX details: live slot-availability feedback while selecting a vehicle type, an editable confirmation field after every automatic detection (so AI error never silently becomes a wrong billing record), and a clear escape hatch to manually correct an automatically-created check-in.

3.2 What should be acknowledged directly, not minimized, if asked

- Admin authentication is a hardcoded placeholder, not real security, and the project's own documentation says so.
- The “dynamic pricing” chart is a visual demonstration, not a live, data-driven feature.
- There is no automated test suite.
- Multi-device, simultaneous real-time sync is not implemented; the current sync model is closer to “each session pushes and pulls on its own schedule.”
- Payment confirmation is a manual attendant action with no automated verification against an actual payment event.
- A real API token was found exposed in a file meant to be safely committed, and needs to be rotated regardless of submission timing.

3.3 Suggested framing for the panel

If asked to summarize the project's state in one statement, a defensible and accurate framing is: this is a functionally complete proof-of-concept with production-quality engineering in its core technical pipeline (camera capture, cloud ANPR, fee calculation, payment QR generation, cloud data sync), and a clearly identified, honestly-documented set of next steps — primarily around authentication, automated testing, and real-time multi-device sync — before it would be ready to handle a live facility's actual paying customers and revenue without supervision.

REMINDER

Rotate the Plate Recognizer API token found in .env.example before or immediately after this submission, independent of how the presentation itself goes. This is a five-minute action in the Plate Recognizer dashboard and meaningfully reduces real-world exposure.